

How to Use This Coding Guide

The coding round is reported to have ~2 LeetCode-easy problems.

Organised by pattern (hashing, stack, arrays/two-pointers). Every Python solution is executed and verified.

Authenticity labels appear beside each problem.

Patterns covered: Hashing (3), Stack (1), Arrays / Two Pointers (3)

Pattern: Hashing

Problem 1: Two Sum

Difficulty: Easy | Pattern: Hash Map | Authenticity: Frequently Repeated

Problem Statement

Return indices of the two numbers adding up to the target.

Input Format

Line 1: array. Line 2: target.

Output Format

Two indices, space-separated.

Constraints

$2 \leq n \leq 10^4$

Sample Input

```
3 2 4
6
```

Sample Output

```
1 2
```

Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i

i, j = two_sum([3, 2, 4], 6)
print(i, j)
```

Step-by-Step Explanation

Store indices; when the complement appears, return the pair. $2 + 4 = 6$.

Dry Run

at 4, complement 2 seen -> (1,2)

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 2: Contains Duplicate

Difficulty: *Easy* | Pattern: *Set* | Authenticity: *Community Reported*

Problem Statement

Return True if any value appears at least twice.

Input Format

Line 1: n. Line 2: n integers.

Output Format

True or False.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
5
1 2 3 1 4
```

Sample Output

```
True
```

Python Solution

```
def contains_duplicate(nums):
    return len(set(nums)) != len(nums)

print(contains_duplicate([1, 2, 3, 1, 4]))
```

Step-by-Step Explanation

A set drops duplicates; a smaller set size means a duplicate existed.

Dry Run

```
set size 4 != list size 5 -> True
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 3: Valid Anagram

Difficulty: *Easy* | Pattern: *Frequency Counting* | Authenticity: *Community Reported*

Problem Statement

Decide whether two strings are anagrams.

Input Format

Line 1: first string. Line 2: second string.

Output Format

True or False.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
listen  
silent
```

Sample Output

```
True
```

Python Solution

```
def is_anagram(a, b):  
    if len(a) != len(b):  
        return False  
    counts = {}  
    for ch in a:  
        counts[ch] = counts.get(ch, 0) + 1  
    for ch in b:  
        if counts.get(ch, 0) == 0:  
            return False  
        counts[ch] -= 1  
    return True  
  
print(is_anagram('listen', 'silent'))
```

Step-by-Step Explanation

Count first string, decrement for second; all cancel -> anagram.

Dry Run

```
listen/silent letters cancel -> True
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Pattern: Stack

Problem 1: Valid Parentheses

Difficulty: **Easy** | Pattern: **Stack** | Authenticity: **Community Reported**

Problem Statement

Decide whether a bracket string is validly matched.

Input Format

A single line of brackets.

Output Format

True or False.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
()[]{}
```

Sample Output

```
True
```

Python Solution

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack

print(is_valid('()[]{}'))
```

Step-by-Step Explanation

Push openers; each closer must match the top; end with an empty stack.

Dry Run

each pair matches; empty stack -> True

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Pattern: Arrays / Two Pointers

Problem 1: Best Time to Buy and Sell Stock

Difficulty: **Easy** | Pattern: **One Pass** | Authenticity: **Community Reported**

Problem Statement

One buy and one later sell; maximise profit (0 if none).

Input Format

Line 1: n. Line 2: n prices.

Output Format

Maximum profit.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
6
7 1 5 3 6 4
```

Sample Output

```
5
```

Python Solution

```
def max_profit(prices):
    min_p = float('inf')
    best = 0
    for p in prices:
        min_p = min(min_p, p)
        best = max(best, p - min_p)
    return best

print(max_profit([7, 1, 5, 3, 6, 4]))
```

Step-by-Step Explanation

Track the min price so far; best profit is price minus that min. Buy 1 sell 6.

Dry Run

```
min->1; best 6-1=5
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 2: Merge Two Sorted Arrays

Difficulty: **Easy** | Pattern: **Two Pointers** | Authenticity: **Community Reported**

Problem Statement

Merge two sorted arrays into one sorted array.

Input Format

Line 1: first array. Line 2: second array.

Output Format

Merged array, space-separated.

Constraints

$0 \leq \text{sizes} \leq 10^5$

Sample Input

```
1 3 5
2 4 6
```

Sample Output

```
1 2 3 4 5 6
```

Python Solution

```
def merge_sorted(a, b):
    i = j = 0
    merged = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            merged.append(a[i]); i += 1
        else:
            merged.append(b[j]); j += 1
    merged.extend(a[i:]); merged.extend(b[j:])
    return merged

print(' '.join(map(str, merge_sorted([1, 3, 5], [2, 4, 6]))))
```

Step-by-Step Explanation

Take the smaller front element each step; append the remainder. Merge-sort step.

Dry Run

1, 2, 3, 4, 5, 6

Time Complexity: **O(m+n)** | Space Complexity: **O(m+n)**

Problem 3: Maximum Subarray Sum (Kadane)

Difficulty: **Medium** | Pattern: **DP / Greedy** | Authenticity: **Community Reported**

Problem Statement

Maximum sum of any contiguous subarray.

Input Format

Line 1: n. Line 2: n integers.

Output Format

Maximum subarray sum.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Sample Output

```
6
```

Python Solution

```
def max_subarray(arr):
    best = cur = arr[0]
    for x in arr[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

Step-by-Step Explanation

Extend or restart the running sum; track the best. [4,-1,2,1] -> 6.

Dry Run

best peaks at 6

Time Complexity: **O(N)** | Space Complexity: **O(1)**

BI3 Technologies

Coding Problems (LeetCode-Easy, Pattern-wise)

Applied Role(s):

Data Engineer - Trainee

Online Assessment Preparation Material

Version 1.1 • Last Updated: July 2026

Every question carries an authenticity label. See the legend on the next page.

Table of Contents

How to Use This Coding Guide	1
Pattern: Hashing	1
Pattern: Stack	3
Pattern: Arrays / Two Pointers	4
Authenticity Legend & How to Read This Document	9

Authenticity Legend & How to Read This Document

Every question and problem in this document is labelled with an **authenticity classification** so you know how closely it maps to what candidates have actually reported. Labels are assigned conservatively based on research; authored practice questions are never presented as real exam questions.

Label	Meaning
Verified Previous-Year	The specific problem/pattern is documented from actual reported assessment experiences.
Frequently Repeated	The topic or pattern type is documented as recurring across multiple hiring cycles.
Community Reported	A widely shared, role-relevant pattern reported across placement communities and forums.
Research-Backed Company Style	Authored to match the researched difficulty, style and pattern - practice, not a real past question.

The correct option in every MCQ is highlighted in green and also restated below the question. Study the pattern, not just the answer.